Assignment 9

TDT4171 — Artificial Intelligence Methods

March 2026

Information

- **Delivery deadline: March 30, 2026 by 23:59**. No late delivery will be graded! Deadline extensions will only be considered for extraordinary situations such as family or health-related circumstances. These circumstances must be documented, e.g., with a doctor's note ("legeerklæring"). Having a lot of work in other classes is not a legitimate excuse for late delivery.
- Cribbing ("koking") from other students is not accepted, and if detected, will lead to immediate failure of the course. The consequence will apply to both the source and the one cribbing.
- Students can **not** work in groups. Each student can only submit a solution individually.
- Relevant reading for this assignment: Chapter 23. Reinforcement Learning (the parts in the curriculum found on Blackboard "Sources and syllabus") of Artificial Intelligence: A Modern Approach, Global Edition, 4th edition, Russell & Norvig.
- For help and questions related to the assignment, **ask the student assistants during the guidance hours**. The timetable for guidance hours can be found under "Assignments" on Blackboard. For other inquires, an email can be sent to tdt4171@idi.ntnu.no.
- Deliver your solution on Blackboard. Please upload your assignment as one PDF report and one source file containing the code (i.e., one .py file) as shown in Figure 1.

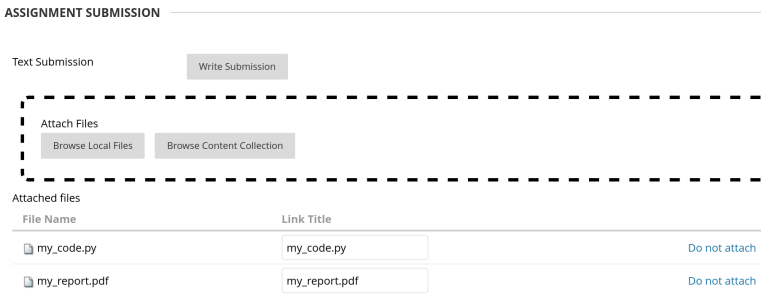


Figure 1: Delivery Example

Assignment Information

In this assignment, you will get familiar with the Python libraries Gymnasium for Reinforcement Learning (RL) problem environments, as well as Stable-Baselines3 for readily implemented RL algorithms.

You will need to install both libraries, see installation guides for Gymnasium here and for Stable-Baselines3 here. See here for a Python installation a guide.

You can find a simple code example here.

Exercise

You will work with the Frozen lake environment.

This environment is imported with the following lines

```
import gymnasium as gym

env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=True).
```

The Frozen lake environment is a grid of either size 4x4 (for `map_name="4x4"`) or 8x8 (for `map_name="8x8"`), where each square is either start, ice, hole or goal. See Figure 2. To reach the goal, the agent needs to avoid the holes.

The action space is {Left, Down, Right, Up}. If `is_slippery` is set to `False`, the environment is deterministic, and the agent always moves in the direction of the chosen action. If `is_slippery = True`, walking on the ice is unpredictable. The player will move in the intended direction with a probability of 1/3, or in either perpendicular direction with a probability of 1/3 each. If the agent chooses the action Down, there is a 1/3 chance of moving down, 1/3 chance of moving to the left and a 1/3 chance of moving to the right, etc.



Figure 2: The frozen lake environment, size 4x4. Top left is the start state, and bottom right is the goal state. States are represented in the observation space by integers $0-15$, as given by $r*4+c$ for the row r and column c of each state ($0 \leq r, c \leq 3$). I.e., start is 0, the agent is in 1, the holes are 5, 7, 11, 12 and goal is 15.

You should try the following settings:

1. map_name="4x4", is_slippery=False
2. map_name="4x4", is_slippery=True
3. map_name="8x8", is_slippery=True

The rewards of the Frozen Lake environments are set such that you get a reward of 1 if you reach the goal and a reward of 0 otherwise, i.e. all states have a 0 reward except for the goal. The episode ends when the agent either falls into a hole or reaches the goal, or if the length of the episode reaches 100 (4x4)/ 200 (8x8) steps.

The task is to train an agent with a suitable algorithm from Stable-Baselines3 such that the following mean rewards are achieved for each environment setting:

1. 1.0
2. 0.7
3. 0.5

You can for instance use a Deep Q-Network (DQN):

```
from stable_baselines3 import DQN

model = DQN("MlpPolicy", env, verbose=1, exploration_fraction=0.1)
```

Here, `exploration_fraction` sets the fraction of the learning process where exploration is prioritized. You might want to change this to improve policy learning. See this page for other hyperparameters to consider, or this page for other RL algorithms.

Mean reward can be evaluated using the following lines of code:

```
from stable_baselines3.common.evaluation import evaluate_policy

mean_rewards, _ = evaluate_policy(model, model.get_env(), n_eval_episodes=100)
```

where `model` is your trained agent.

The PDF report should include:

1. Document the best mean reward reached for each environment setting (Include numbers in the PDF and deliver your code for each of the environment+agent pairs).
2. For the agents you have trained on the two 4x4 size environments, visualize and document the learned policies. Compare the policy for the deterministic environment (`is_slippery = False`) with that of the stochastic environment (`is_slippery = True`). Are they reasonable?

   Consider the learned policies in the following states:

   - in State 1 (where the agent is shown in Figure 2, one step right from the start)
   - in State 6 (second row, third column: the ice square with a hole both left and right)

   Have the respective agents learned the optimal policy here? Explain what the optimal policy is, how they differ between stochastic and deterministic environment and why, for these two states.